MECHTRON / SFWRENG 3K04: Software Development

Project Deliverable 1 – Fall 2026

McMaster University

Due Date: Friday, October 9th, 2026 11:59 PM

This handout applies to both parts of the project. Each sub-team will make their own submission as explained below.

1 Deliverable 1 Purpose

The purpose of this first deliverable is to establish *what* your system must do and for *whom*, and document and model it clearly. You will identify stakeholders, elicit, document, and specify requirements, express them as user stories with acceptance criteria, model the system's structure and behaviour with UMLs, and draw a high-level architecture diagram showing the major components and how they relate to each other, to the users, to the hardware, and to the partner system. This deliverable is the foundation the rest of the project builds on. Get excited :)

2 Grading

This deliverable is 10% of your final course grade. You will be assessed in two ways: a technical demonstration to the TAs during your lab (worth 70%) and the evaluation of your submission file(s) (worth 30%).

Deliverable 1 will be demo'ed during your lab session the week of **October 5th, 2026**. The submission files will be due at the end of the week (**Friday, October 9th, 2026 11:59 PM**).

3 Technical Demonstration (70%)

The students will walk through the requirements they selected, user stories, and UML models, and answers questions. Every student must be able to speak to every requirement, stories, and diagram. The TAs will choose who answers.

What will the TAs look for:

- Requirements are clear, unambiguous, verifiable, and traceable to a stakeholder or scenario.
- User stories are well-formed (role ->goal ->benefit) with meaningful acceptance criteria that a tester could check.
- UML models cover both structure (domain/class model) and behaviour (use case, activity, state machine), use proper notation, and are consistent with the written requirements and user stories.
- The team has considered the ethical dimension of the system and can point to how it shaped requirements (e.g., patient safety, privacy of medication records, controlled access, accountability/auditability).

- The team can move between the three views: a requirement, the story that captures it, and the diagram that shows it.
- Individual members, not just one spokesperson, understand the work.
- The team can absorb an on-the-spot change (a new actor, a new story) and reason about its effect on the requirements and models.

4 Submission File(s) (30%)

A well-structured requirements document (with the UML models embedded). This is the evidence the TAs mark the demonstration against, and it should read as professional documentation. While the full requirements document is due after your technical demonstration, you should have it ready (about 95% done) during the demonstration as the TAs will ask you questions about it.

Structure of the requirements document:

- **Introduction and system context:** Introduction to the document, summary of project purpose and scope.
- **Functional requirements:** organized by capability, each uniquely identified, atomic, unambiguous, and verifiable. Cover your project part's capabilities:
  - For those working on the ADC: authentication, inventory management, dispensing, event logging, fault detection, operational modes, external communication.
  - For those working on the monitoring system: ADC communication, inventory supervision, alarm management, historical data, reporting, operator dashboard, cabinet-health monitoring.
- **Non-functional requirements:** safety, security, reliability, availability, performance, traceability/auditability, usability.
- **User stories:** the requirements expressed as stories in the form *"As a <role>, I want <goal> so that <benefit>"*, each with acceptance criteria (e.g., Given/When/Then). Organize into themes where useful; keep stories aligned to the INVEST qualities (independent, negotiable, valuable, estimable, small, testable).
- **High-level architecture diagram:** a block/component diagram showing the major components of your system and what relates with what such as the internal components, the users/actors, the hardware, and the partner system, with labelled connections indicating what is exchanged across each (and in which direction). Mark the ADC <– >Monitoring system boundary clearly. Keep this at a conceptual level so it shows major components and relationships, not the detailed module decomposition and interfaces (those will come in Deliverable 2).
- **UML models — structure and behaviour:**
  - **Conceptual domain / class model (Structure):** the key domain entities and their relationships (attributes/behaviour optional at this stage; this is analysis-level, not detailed design).

  - **Use case diagram(s) (Behaviour):** actors and use cases, with <<*include* >>/ <<*extend* >> relationships, covering the main capabilities.
  - **Activity diagram(s) (Behaviour):** the flow of the key operational scenarios (the normal path plus off-normal paths such as hardware fault or communication loss). Pick at least 4 scenarios.
  - **State machine diagram(s) (Behaviour):** the state-dependent behaviour of the system, e.g. the operational modes (normal / maintenance / fault) and the dispensing or alarm lifecycle, with the events that trigger each transition and the handling of invalid or fault events.

- **Traceability table:** Requirement ID ->user story ->component (on the architecture diagram) ->stakeholder or source, so the whole set of views stays connected.
- **Assumptions and decisions:** Summary of every assumption made to fill in gaps, and the decisions that follow and shaped specific requirements. Include accessibility and ethical considerations (e.g., patient safety, privacy of medication and patient-related records, controlled/authorized access, etc.)

5 Tips for Success

- Write every requirement and acceptance criterion so a TA could test it. If you can't describe how to verify it, rewrite it.
- Keep the views consistent: each user story should trace to a requirement and appear in a use case; each scenario should have an activity diagram; state machine states and transitions should match the modes and events named in your requirements; and every actor, external system, and hardware element in your diagrams should appear on the high-level architecture diagram. Inconsistencies are the fastest way to lose demo marks.
- Keep the architecture diagram high-level: show major components and what flows between them, not internal module design. Save the detailed decomposition, interfaces, and USES relation for Deliverable 2.
- Use proper UML notation such as actors, system boundary, <<*include* >>/ <<*extend* >>, correct activity-diagram control nodes (decisions, forks/joins). Neatness signals understanding.
- Make your assumptions explicit! The description of each part of the project deliberately leaves gaps for you to fill. At the same time, seek clarification when needed from the TAs and/or *ReQuest*.
- Keep a stable requirement/story ID scheme now; you will trace design and tests back to these IDs during the future deliverable.
- Rehearse as a sub-team before the demo so any member can defend any aspect of the document.

6 Submission Guidelines

Each sub-team will see an assignment folder to upload the requirements document on Avenue. Allow yourself enough time before the deadline in case of any technical difficulties.